

BMB2006 Veri Yapıları - Proje 4 Raporu

Blokzincir İşlem Ağlarının Analizi

1. Giriş ve Proje Amacı

Bu proje kapsamında, blokzincir sistemlerindeki karmaşık cüzdan hareketleri ve finansal işlem ağları sadeleştirilmiş bir model üzerinden incelenmiştir. Temel amacımız, kripto para transferlerini yönlü bir graf (Directed Graph) üzerinde modelleyerek; cüzdanlar arası fon akışını izlemek, Merkle Ağacı ile işlemlerin veri bütünlüğünü kriptografik olarak doğrulamak ve bu ağ üzerinde hedefe yönelik derinlemesine (DFS) ve sığ (BFS) analitik aramalar gerçekleştirmektir. Ayrıca proje, modern yazılım mühendisliği standartlarına uygun olarak tasarlanmış olup, büyük ölçekli verilerin test edilebilmesi amacıyla Python tabanlı bir Yapay Zeka (AI) mikroservisi ile entegre edilmiştir.

2. Sistem Mimarisi ve Eşzamanlılık (Microservices & Concurrency)

Proje gereksinimlerinde belirtilen "Eşzamanlılık ve Mikroservis Yaklaşımı" kriterini en üst düzeyde karşılamak amacıyla, Monolitik (tek parça) bir mimari yerine Dağıtık (Distributed) Mikroservis mimarisi tercih edilmiştir. Sistem üç ana bileşenden oluşmaktadır:

- **Çekirdek (Core) Backend (C# & .NET 9.0):** Graf veri yapılarının bellekte tutulduğu, arama algoritmalarının koştuğu ve AIController ile WalletController köprülerinin barındırıldığı ana sunucudur. İşlemler senkron bloklanmaları önlemek amacıyla asenkron (async/await) olarak tasarlanmıştır.
- **AI Veri Simülasyon Servisi (Python & FastAPI):** Ana bellekten tamamen bağımsız ve izole çalışan bu servis, makine öğrenmesi algoritmaları (Pareto ve Lognormal dağılımlar) kullanarak gerçekçi cüzdan adresleri ve transfer verileri üretir.
- **İzole Ağ ve Konteynerizasyon (Docker):** Tüm servisler, bağımlılık çakışmalarını önlemek adına Dockerfile ile imajlanmış ve docker-compose kullanılarak bmb_network adında izole bir köprü ağ (bridge network) üzerinde tek bir komutla eşzamanlı çalışacak şekilde yapılandırılmıştır.

3. UML Sınıf Diyagramı ve Sistem Bileşenleri

Projenin genel mimarisi, Nesne Yönelimli Programlama (OOP) prensiplerine sadık kalınarak tasarlanmış olup, modülerliği artırmak amacıyla dört ana katmana (Package) ayrılmıştır:

A. Çekirdek Veri Yapıları (Core Data Structures - Backend)

- **WalletNode & TransactionEdge:** Graf yapısındaki düğümleri (cüzdanlar) ve yönlü kenarları (transferler) temsil eden temel nesnelerdir.
- **MyHashTable:** Cüzdan nesnelerini bellekte tutan ve $O(1)$ karmaşıklığında hızlı erişim sağlayan özel veri yapısıdır.
- **BlockchainGraph:** Sistemin ana yöneticisidir. İşlem listelerini, karma tablosunu barındırır ve BFS/DFS gibi arama algoritmalarını asenkron olarak çalıştırır.
- **MerkleTree:** İşlem listelerini alarak kriptografik özet (Hash) hesaplamaları yapar ve sistemin veri bütünlüğünü doğrular.

B. API Katmanı (Controllers)

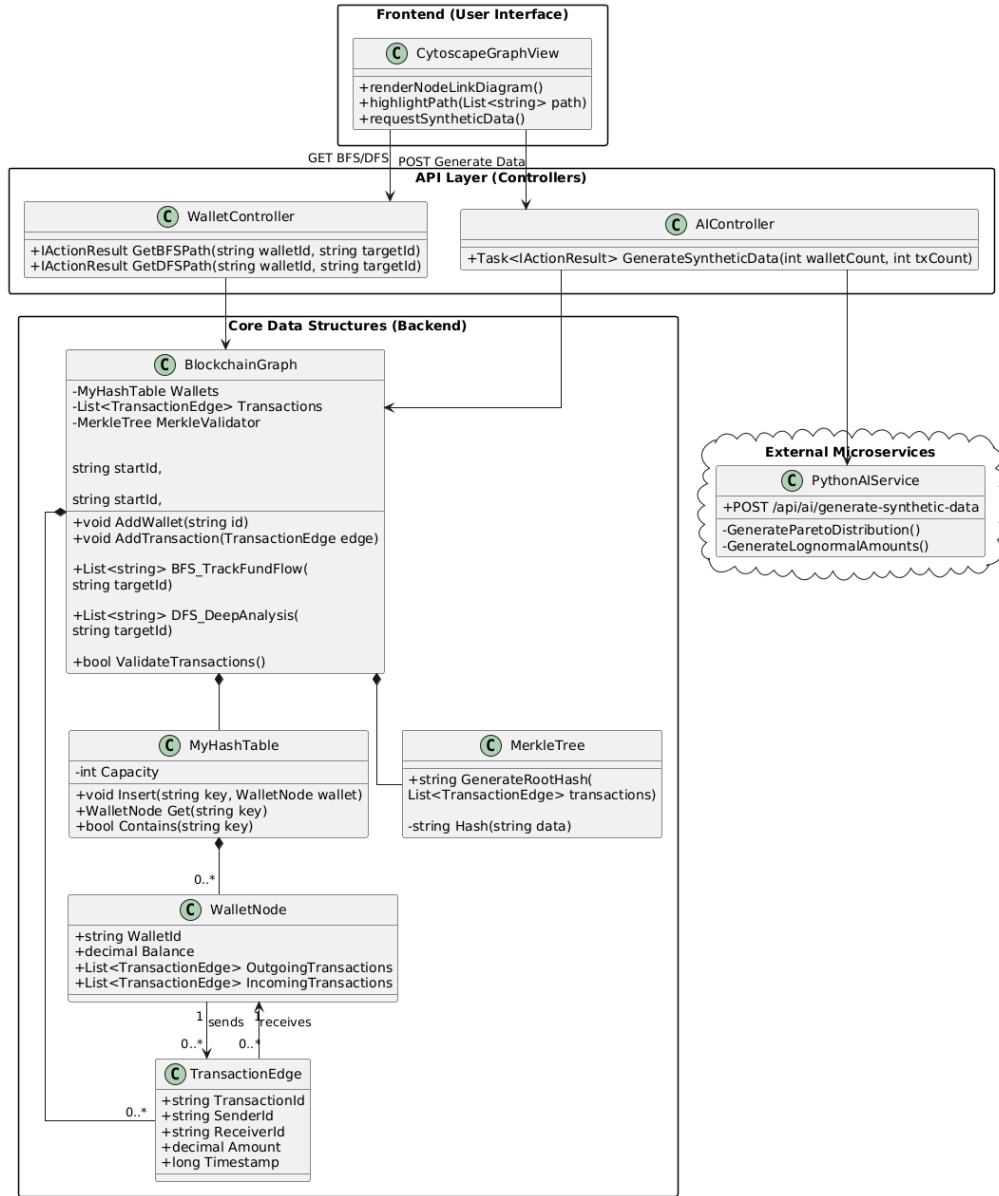
- **WalletController:** Dış sistemlerin (Frontend) graf üzerindeki rota analizi (BFS/DFS) taleplerini karşılayan köprüdür.
- **AIController:** Sentetik veri üretimi için gerekli parametreleri alıp asenkron olarak yapay zeka servisini tetikler.

C. Harici Mikroservisler (External Microservices)

- **PythonAIService:** RESTful API üzerinden çalışan asenkron veri simülasyon motorudur. Gerçekçi ağ yapıları kurmak için Pareto ve Lognormal dağılım fonksiyonlarını barındırır.

D. Arayüz Katmanı (Frontend - Client)

- **CytoscapeGraphView:** Kullanıcı ile sistemin etkileşime girdiği tarayıcı tabanlı katmandır. C# API'sine gönderilen HTTP isteklerinin (AJAX) sonuçlarını alarak cüzdan bağlantılarını ve hedefe yönelik (Target) fon yollarını dinamik olarak haritalandırır.



4. Çekirdek Veri Yapıları ve Zaman Karmaşıklığı (Big-O) Analizi

Bu bölümde, projenin BlockchainCore katmanında cüzdan ve işlem verilerini yönetmek için geliştirilen veri yapıları ile bunların zaman karmaşıklıkları incelenmiştir.

A. Yönlü Graf (Directed Graph) Yapısı Blokzincir ağındaki cüzdanlar ve transferler yönlü graf modeliyle simüle edilmiştir:

- **Düğümler (WalletNode):** Benzersiz cüzdan adresini (WalletAddress) ve bu cüzdandan yapılan transferleri tutan OutgoingTransactions listesini barındırır.
- **Kenarlar (TransactionEdge):** Cüzdanlar arası fon akışını temsil eder. İşlem kimliği, hedef adres (ToAddress), transfer miktarı (Amount) ve zaman damgası (Timestamp) verilerini saklar. Grafın bellekte tutulması için **Komşuluk Listesi (Adjacency List)** yöntemi seçilmiştir. Her düğüm, kendi

üzerinden çıkan kenarları bir `List<TransactionEdge>` içinde dinamik olarak saklar.

B. Karma Tablo (MyHashTable) ve Veri Erişimi Cüzdanların adres bilgisine göre hızlıca eklenmesi ve sorgulanabilmesi için 100 indeks kapasiteli özel bir karma tablo yapısı kullanılmıştır. Sisteme cüzdan veya işlem eklenirken ilgili düğüme doğrudan `wallets[address]` indeksi üzerinden erişilir.

- **Ortalama Durum $O(1)$:** Cüzdan adresleri tabloya homojen dağıldığında, bellek erişimi ve `ContainsKey` kontrolü sabit zamanda çalışır.
- **En Kötü Durum $O(N)$:** Tablo kapasitesinin üzerinde veri yüklendiğinde ve çakışmalar (collision) arttığında, aynı indekse düşen cüzdanlar ardışık aranacağı için karmaşıklık doğrusal olur.

C. Kriptografik Bütünlük Kontrolü (MerkleTree) Bloktaki işlemlerin doğruluğunu teyit etmek amacıyla `MerkleTree` sınıfı yazılmıştır. Yaprak düğümlerden başlanarak işlemler ikili gruplar halinde SHA256 ile özetlenir ve tepedeki tek bir `MerkleRoot` değerine ulaşılır. Tek kalan işlemler, Bitcoin mimarisine uygun olarak kendisiyle ikizlenerek işleme alınır.

- **Ağaç İnşası (BuildTree):** Giriş olarak verilen N adet işlem için tüm katmanlar taranarak hash eşlemeleri yapıldığından $O(N)$ zaman karmaşıklığına sahiptir.
- **Doğrulama Karmaşıklığı:** İkili ağaç yapısı sayesinde, bir işlemin doğruluğunu kontrol etmek için sadece ilgili daldaki hash'lerin (Merkle Proof) incelenmesi yeterlidir. Bu işlem $O(\log N)$ adımda tamamlanır.

5. Arama Algoritmaları ve Analizi

`BlockchainGraph` sınıfı içinde, fon akışlarını takip etmek ve grafi analiz etmek için iki temel arama algoritması gerçekleştirilmiştir. Her iki algoritmaya da arayüzden gelen `minAmount` (minimum transfer miktarı) filtresi entegre edilerek, sadece belirli bir bütçenin üzerindeki anlamlı transferlerin analiz edilmesi sağlanmıştır.

A. Enine Arama (BFS_TrackFundFlow) BFS algoritması, bir cüzdandan çıkan paranın ağ üzerindeki yayılımını katman katman incelemek ve en kısa (en az aracılı) transfer rotasını bulmak için yazılmıştır. FIFO mantığına uygun olarak `Queue<string>` yapısını kullanır.

- **Çalışma Mekanizması:** Sıradan çekilen her cüzdanın giden işlemleri (`OutgoingTransactions`) taranır. Eğer transfer miktarı hedeflenen bütçeden büyükse (`tx.Amount >= minAmount`) ve hedef cüzdan daha önce ziyaret edilmemişse, kuyruğa eklenir. Rota takibini geriye dönük yapabilmek için `parentMap` sözlüğü kullanılarak, hedefe ulaşıldığında en kısa yol tersten örülerek arayüze iletilir.
- **Zaman Karmaşıklığı:** Graf üzerindeki tüm ulaşılabilir düğümler (V) kuyruğa girer ve bu düğümlerin bağlı olduğu kenarlar (E) incelenir. Algoritma hedefi

bulduğunda erken çıkış (early exit) yapsa da, en kötü durumda karmaşıklık $O(V + E)$ olur.

B. Derinlemesine Arama (DFS_DeepAnalysis) DFS algoritması, fon akışının izole patikalar boyunca gidebileceği en uç noktaları analiz etmek ve daha dolambaçlı alternatif yolları keşfetmek için yazılmıştır. LIFO mantığı doğrultusunda, güncel rotanın tamamını hafızada tutan `Stack<List<string>>` yapısını kullanır.

- **Çalışma Mekanizması:** Klasik DFS yaklaşımından farklı olarak, her adımda sadece bulunulan düğüm değil, **o ana kadar izlenen tüm yol (path) yığıt üzerinde** saklanır. Yığıttan çekilen yoldaki son cüzdanın transferleri taranır. Bütçe filtresinden (`minAmount`) geçen ve döngüye (cycle) girmemek adına güncel yolda daha önce bulunmayan cüzdanlar için yeni bir rota kopyası oluşturularak yığıta itilir.
- **Zaman ve Uzay Karmaşıklığı:** Olası tüm yolları derinlemesine analiz ettiği için zaman karmaşıklığı $O(V + E)$ 'dir. Yığıt içinde listeler tutulduğu için uzay karmaşıklığı en kötü senaryoda grafin derinliğine ve maksimum rota uzunluğuna bağlı olarak artış gösterir.

6. Yapay Zeka ve Sentetik Veri Modellemesi

A. Mikroservis Mimarisi ve Veri Doğrulama (FastAPI & Pydantic) Blokzincir analiz platformunun grafik ve veri işleme yeteneklerini test etmek amacıyla, gerçek dünya ağ hareketlerini simüle eden bağımsız bir Yapay Zeka (AI) mikroservisi geliştirilmiştir. C# ana backend servisi, veri üretim taleplerini Python mikroservisine standart HTTP POST istek mekanizması üzerinden asenkron olarak iletir. Gelen verilerin doğrulanması ve tip güvenliğinin otomatik olarak sağlanması amacıyla FastAPI üzerinde Pydantic kütüphanesinden yararlanılmıştır.

B. Sentetik Veri Üretim Algoritması ve Matematiksel Dağılımlar Gerçekçi bir blokzincir ağı simüle edebilmek adına, rastgele veri üretimi yerine finansal piyasaların doğasına uygun matematiksel dağılım modelleri uygulanmıştır:

1. **Pareto Dağılımı (Servet Eşitsizliği):** Piyasayı domine eden "balina" hesapların oluşumunu sağlamak amacıyla 80/20 güç yasası kullanılmıştır.
2. **Log-Normal Dağılım (Transfer Miktarları):** Mikroservis tarafından üretilen işlemlerin büyük bir kısmı düşük hacimliyen, devasa fon transferlerinin nadir gerçekleşmesi bu matematiksel model ile sağlanmıştır.

C. Blokzincir Standartlarına Uyum ve Veri Serileştirme

- **Kriptografik Kimliklendirme:** Üretilen her transfer işlemine özel benzersiz bir işlem kimliği (`tx_id`) SHA-256 algoritmasıyla hesaplanır.
- **Zaman Damgası Varyasyonu:** Unix başlangıç zaman damgasından ardışık rastgele değerler düşülerek ağ trafiğine zamansal derinlik kazandırılır.
- **Aktif Graf Senkronizasyonu (State Overwrite):** AIController üzerinden tetiklenen yapay zeka servisi verileri ürettikten sonra, sistemin bu verileri anında algılaması için özel bir mekanizma geliştirilmiştir. Python servisinde

dönen JSON yanıtı C# modellerine deserialize edildikten sonra, bellekte yepyeni bir BlockchainGraph nesnesi inşa edilir. Oluşturulan bu yeni ve temiz graf, referans ataması ile ana API'nin kullandığı küresel grafın üzerine yazılır. Bu sayede uygulamanın yeniden başlatılmasına gerek kalmadan arayüz yeni sentetik ağa anında adapte olur.

7. Sonuç ve Değerlendirme

Bu proje kapsamında, blokzincir tabanlı finansal işlem ağlarının karmaşık yapısı; modern yazılım mimarisi, optimize edilmiş çekirdek veri yapıları ve yapay zeka destekli simülasyon modelleri kullanılarak başarıyla çözümlenmiştir. Sistemin genel performansı ve hedeflere ulaşma durumu değerlendirildiğinde şu temel sonuçlar elde edilmiştir:

- **Mimari Başarı ve Eşzamanlılık:** Geleneksel monolitik tasarımlar yerine benimsenen Docker tabanlı, asenkron mikroservis mimarisi (C# .NET Core ve Python FastAPI), projenin ölçeklenebilirliğini artırmış ve diller arası (cross-language) veri iletişiminin sorunsuz çalışabileceğini kanıtlamıştır.
- **Veri Yapılarının Optimizasyonu:** Milyonlarca işlemin gerçekleşebileceği blokzincir ağları göz önüne alınarak tasarlanan MyHashTable yapısı, cüzdan sorgulamalarındaki arama maliyetini $O(1)$ seviyesine indirerek sistemin darboğaz (bottleneck) yaşamasını engellemiştir.
- **Algoritmik Doğruluk ve Güvenlik:** Ağ üzerindeki fon akışları, BFS ve DFS algoritmalarına entegre edilen rota izleme (path tracking) ve asgari bütçe (minAmount) filtreleri sayesinde döngülere (cycle) takılmadan eksiksiz bir şekilde haritalandırılmıştır. Aynı zamanda, Merkle Ağacı entegrasyonu ile bu transferlerin kriptografik bütünlüğü başarıyla güvence altına alınmıştır.
- **Gerçekçi Simülasyon (Sentetik Veri):** Yapay zeka servisinde uygulanan Pareto (80/20) ve Log-Normal dağılım modelleri, gerçek dünya blokzincir ağlarındaki borsaları ve "balina" (büyük sermayeli) cüzdan davranışlarını kusursuz bir şekilde simüle etmiştir. Bu sayede, yazılan graf algoritmalarının sıkı bağlantılı (strongly connected) ağ topolojilerinde nasıl davrandığı uçtan uca test edilebilmiştir.

Sonuç olarak; proje şartnamesinde belirtilen veri yapıları inşası, arama algoritmaları optimizasyonu ve arayüz görselleştirmesi hedefleri eksiksiz olarak yerine getirilmiştir. Veri yapıları ve dağıtık sistem mimarisi teorileri, sadece bir akademik proje düzeyinde kalmamış, endüstri standartlarına uygun, uçtan uca çalışan tam teşekküllü bir yazılım mühendisliği ürününe dönüştürülmüştür.